

Project 1: The LogicScript Optimizing Compiler

1. Background

Every modern software application relies on a compiler or interpreter to translate human-readable code into executable machine instructions. This translation and optimization process is deeply rooted in discrete mathematics. Compilers use **recursive definitions** to parse syntax, **propositional logic** to evaluate control flow, and **logical equivalences** to mathematically optimize code before it runs.

In this project, you will transition from solving discrete math problems on paper to building a complete, four-phase optimizing compiler for a custom micro-language called **LogicScript**.

2. Objective

You will design a software pipeline in Python that reads a multi-line text file written in LogicScript, validates its structural integrity, mathematically simplifies its logic for efficiency, and executes its instructions using a state environment.

This project requires you to apply the principles of Propositional Logic (Chapter 2), Recursive Definitions (Chapter 5), and Sets/Relations (Chapter 6 & 8).

3. The LogicScript Grammar & Lexicon

3.1 The Lexical Alphabet (Valid Tokens)

To successfully parse the language, your software must enforce strict recursive definitions. A valid LogicScript program is constructed using only the following terminal symbols:

- **Keywords:** let, if, then, print
- **Booleans:** T, F
- **Operators:** AND, OR, NOT, IMPLIES
- **Symbols:** =, (,)
- **Variables:** Any single lowercase letter (e.g., p, q, x).

3.2 Recursive Syntax Rules

A valid LogicScript program is a sequence of one or more **Statements**.

1. **Expressions:**

- **Base Cases (No parentheses):** The boolean literal T, the boolean literal F, or any single variable (e.g., p) is a valid Expression on its own.
 - **Recursive Cases (Parentheses strictly required):**
 - If E is a valid Expression, then (NOT E) is a valid Expression.
 - If E1 and E2 are valid Expressions, then (E1 AND E2), (E1 OR E2), and (E1 IMPLIES E2) are valid Expressions.
2. **Assignment Statement:** let <Variable> = <Expression>
 3. **Conditional Statement:** if <Expression> then <Statement>
 4. **Print Statement:** print <Variable>
-

4. Task Description: The 4-Phase Pipeline

Phase 1: Lexical Analysis (The Tokenizer)

- Scan the input text file line by line and convert the raw text into a flat list of standardized tokens.
- **Token Mapping Rules:** Your software must act as a translator. It must read the terminal symbols from the file and output **only** the following exact token strings:
 - Keywords: let → 'LET', if → 'IF', then → 'THEN', print → 'PRINT'
 - Booleans: T → 'TRUE', F → 'FALSE'
 - Operators: AND → 'AND', OR → 'OR', NOT → 'NOT', IMPLIES → 'IMPLIES'
 - Symbols: = → 'EQ', (→ 'L_PAREN',) → 'R_PAREN'
 - Variables: Capitalize the letter and prefix with VAR_ (e.g., p → 'VAR_P').
- **Validation:** If the tokenizer encounters an invalid character or unrecognized keyword that is not defined in the Terminal Symbols, it must immediately halt and generate the output in the final result (See Section 5).

Phase 2: Syntax Validation & AST Generation (The Parser)

- Iterate through your flat list of tokens and convert them into a hierarchical **Nested List** (an Abstract Syntax Tree) structure.
- **Validation:** Validate that the tokens strictly follow the Recursive Syntax Rules. If the tokens are out of order (e.g., mismatched parentheses or a missing **then** keyword), the parser must halt and generate the output in the final result.
- **The Target Data Structure:** You cannot simply evaluate a flat sequence of tokens left-to-right because it lacks an understanding of mathematical grouping (Order of Operations). To solve this, your parser must convert the 1D list into a nested list using Prefix Notation (operator first).
 - **NOT operations:** (NOT p) becomes ['NOT', 'VAR_P']

- **Binary operations:** $(p \text{ AND } q)$ becomes `['AND', 'VAR_P', 'VAR_Q']`
- **Statements:** `let x = (p OR T)` becomes `['LET', 'VAR_X', ['OR', 'VAR_P', 'TRUE']]`

(Note: Notice that all parentheses and the equals sign are completely removed in this phase. The structure of the Python lists handles the mathematical grouping.)

Phase 3: The Optimization Pass (Code Simplification)

Your Phase 2 parser will output full statements. Your Phase 3 optimizer must search inside these statements to find and simplify the mathematical expressions without breaking the overall structure.

- Before the program runs, the compiler must mathematically simplify inefficient logic.
- **Static Optimization Rule:** This pass must be completely independent of the program's execution state. Your recursive function must simplify the mathematical structure using *only* the variable names (e.g., `'VAR_P'`) and operators. It must **not** look up or calculate the actual `True/False` values of those variables. Value evaluation belongs strictly in Phase 4.
- Write a function (for example, recursive) that traverses the nested lists and strictly applies the laws we have learned.

(Note: Do NOT apply the Distributive Law, as this expands the syntax tree rather than simplifying it).

Example of Constant Folding (a code optimization technique):

For example, if Phase 2 outputs an assignment statement with inefficient logic:

- **Unoptimized AST:** `['LET', 'VAR_X', ['OR', 'VAR_P', 'TRUE']]`
- **Constant Folding:** Your optimizer recognizes the Universal Bound Law ($p \text{ OR } T \equiv T$) and completely removes the `OR` operation and the variable `p`.
- **Optimized AST:** `['LET', 'VAR_X', 'TRUE']`

(Notice that throughout this process, the optimizer has no idea whether `VAR_P` or `VAR_Q` are actually `True` or `False`. It only manipulates the structure!)

Another Example Optimization:

If your parser (Phase 2) outputs a nested list that represents the logic `NOT (NOT p AND q)`, your Phase 3 optimizer must automatically mutate that list to represent `(p OR NOT q)`.

- **Unoptimized AST:** `['NOT', ['AND', ['NOT', 'VAR_P'], 'VAR_Q']]`
- **Step 1 (De Morgan's):** The outer `NOT` is pushed inward, flipping the `AND` to an `OR`, resulting in:
`['OR', ['NOT', ['NOT', 'VAR_P']], ['NOT', 'VAR_Q']]`
- **Step 2 (Double Negation):** The redundant `NOT`s cancel out, resulting in the final, optimized AST:
Optimized AST: `['OR', 'VAR_P', ['NOT', 'VAR_Q']]`

(Notice that throughout this process, the optimizer has no idea whether `VAR_P` or `VAR_Q` are actually `True` or `False`. It only manipulates the structure!)

Phase 4: Verification & Execution

- **The Safety Net (Equivalence Checker):** Before executing, your engine must mathematically prove the optimization is safe. For every simplified expression, generate a full 2^n truth table comparing the *original* AST and the *optimized* AST.
- **Execution:** Create a "State Dictionary" (a mathematical relation mapping variables to their evaluated truth values). Process the optimized statements line by line, update the dictionary during assignments, and capture any `print` outputs.
- **The Final Deliverable:** Instead of simply printing to the console, your compiler must bundle the generated truth tables, the final state dictionary, and the captured print outputs into a strictly formatted output file. This file will be used by the automated grading system to verify your logic (See Section 5 for the exact formatting requirements).

Example Verification & Execution:

Assume the engine receives the following two statements from Phase 3:

```
['LET', 'VAR_X', ['OR', 'VAR_P', ['NOT', 'VAR_Q']]]
['PRINT', 'VAR_X']
```

- **Step 1: Equivalence Verification**

The system extracts the original and optimized logic for `VAR_X` and generates a 4-row truth table for variables `p` and `q` to mathematically prove they are equivalent:

p	q	Original	Optimized
T	T	T	T
T	F	T	T
F	T	F	F
F	F	T	T

- **Step 2: Dictionary Update & Execution**

Assume the program's State Dictionary currently looks like this: `{'VAR_P': 'TRUE', 'VAR_Q': 'FALSE'}`.

The engine evaluates the assignment expression based on the dictionary: `TRUE OR (NOT FALSE)  $\equiv$  TRUE`.

It updates the dictionary to store the new variable: `{'VAR_P': 'TRUE', 'VAR_Q': 'FALSE', 'VAR_X': 'TRUE'}`.

Finally, it processes the `PRINT` statement and captures the output: `"TRUE"`.

- **Step 3: Final Export**

Instead of printing these results directly to the console, the compiler packages the generated truth table, the final dictionary state, and the captured print outputs into a single output file. (See Section 5 for the exact file structure and formatting).

5. End-to-End Example Trace & Final Deliverable

Your final submission will be a single Python file (e.g., `logic_compiler.py`) containing your compiler pipeline. To function like a true command-line tool, your program must **not** hardcode the

file names. Instead, it must accept the input text file and the output JSON file as command-line arguments.

By associating each step with its original line number, the engine maintains a strict "source map" for automated grading and error reporting. To maintain mathematical consistency, all boolean evaluations must be formatted as the uppercase string literals "TRUE" or "FALSE" (do not use native JSON booleans).

Execution & Input:

The automated grading script will execute your compiler from the terminal using the following format:

```
python logic_compiler.py <input_file> <output_file>
```

For example, to grade your code, the system might run:

```
python logic_compiler.py program.txt compiler_trace.json
```

Your Python script must be designed to read these arguments, open the input file, process the text, and write the final JSON object to the specified output file. For example:

```
import sys
import json

# Ensure correct command-line usage
if len(sys.argv) != 3:
    print("Usage: python logic_compiler.py <input_file> <output_file>")
    sys.exit(1)

input_filename = sys.argv[1]
output_filename = sys.argv[2]

# 1. Read the input file
with open(input_filename, 'r') as file:
    source_lines = file.readlines()

# ... run your 4-phase compiler pipeline ...

# 2. Export the final JSON trace
with open(output_filename, 'w') as out_file:
    json.dump(pipeline_results_dict, out_file, indent=2)
```

A. Successful Compilation (Valid Input)

Assuming the input file contains valid LogicScript, your program must process all four phases.

Example Valid Input (program.txt):

```
let p = T
let q = F
let r = (NOT ((NOT p) AND q))
if r then print p
```

Example Valid Output (compiler_trace.json):

Upon termination, your program must generate exactly this JSON structure:

```

{
  "phase_1_lexer": [
    {"line": 1, "tokens": ["LET", "VAR_P", "EQ", "TRUE"]},
    {"line": 2, "tokens": ["LET", "VAR_Q", "EQ", "FALSE"]},
    {"line": 3, "tokens": ["LET", "VAR_R", "EQ", "L_PAREN", "NOT", " "
      ↪ L_PAREN", "L_PAREN", "NOT", "VAR_P", "R_PAREN", "AND", "VAR_Q",
      ↪ "R_PAREN", "R_PAREN"]},
    {"line": 4, "tokens": ["IF", "VAR_R", "THEN", "PRINT", "VAR_P"]}
  ],
  "phase_2_parser": [
    {"line": 1, "ast": ["LET", "VAR_P", "TRUE"]},
    {"line": 2, "ast": ["LET", "VAR_Q", "FALSE"]},
    {"line": 3, "ast": ["LET", "VAR_R", ["NOT", ["AND", ["NOT", "VAR_P"],
      ↪ "VAR_Q"]]]}],
    {"line": 4, "ast": ["IF", "VAR_R", ["PRINT", "VAR_P"]]}
  ],
  "phase_3_optimizer": [
    {"line": 1, "ast": ["LET", "VAR_P", "TRUE"]},
    {"line": 2, "ast": ["LET", "VAR_Q", "FALSE"]},
    {"line": 3, "ast": ["LET", "VAR_R", ["OR", "VAR_P", ["NOT", "VAR_Q"]]]
      ↪ },
    {"line": 4, "ast": ["IF", "VAR_R", ["PRINT", "VAR_P"]]}
  ],
  "phase_4_execution": {
    "verifications": [
      {
        "line": 3,
        "variables_tested": ["VAR_P", "VAR_Q"],
        "ast_original_column": ["TRUE", "TRUE", "FALSE", "TRUE"],
        "ast_optimized_column": ["TRUE", "TRUE", "FALSE", "TRUE"],
        "is_equivalent": "TRUE"
      }
    ],
    "final_state_dictionary": {
      "VAR_P": "TRUE",
      "VAR_Q": "FALSE",
      "VAR_R": "TRUE"
    },
    "printed_output": [
      {"line": 4, "output": "TRUE"}
    ]
  }
}

```

B. Handling Compilation Errors

The automated grader will also test if your compiler correctly catches bad code by feeding it a `program.txt` file containing intentional errors.

If the input program contains an error, your compiler must **not** crash Python with a standard traceback. Instead, it must gracefully halt the pipeline, output the phases that succeeded, and append an "error" object to the final JSON file.

Example Error Input (program.txt):

Assume the grader runs your compiler against a file containing a missing operand on line 2:

```
let p = T
if (p AND ) then print p
```

Example Error Output (compiler_trace.json):

The program must catch the syntax error, halt before moving to Phase 3, and export this JSON to the output file:

```
{
  "phase_1_lexer": [
    { "line": 1, "tokens": ["LET", "VAR_P", "EQ", "TRUE"] },
    { "line": 2, "tokens": ["IF", "L_PAREN", "VAR_P", "AND", "R_PAREN", "
      ↪ THEN", "PRINT", "VAR_P"] }
  ],
  "error": {
    "phase": "phase_2_parser",
    "line": 2
  }
}
```

6. Team Organization & Deliverables

Team Organization

- **Group Size:** Students will work in assigned teams of **3 members**.
- **Shared Comprehension:** While teams are highly encouraged to divide technical responsibilities (e.g., building the Lexer, Parser, Optimizer, and Execution Engine), *every* member must maintain a comprehensive understanding of the entire compiler pipeline.
- **Unified Submission:** Each team is required to submit one collective final project.

Deliverables

Each group must submit the following three components:

- **The Codebase:** A well-organized, modular Python file. You may **not** use any external parsing or AST libraries. It must include a comprehensive `README.md` containing clear instructions for execution.
- **Technical Brief (2–3 Pages):** A concise, academic-style document summarizing your compiler's architecture. The brief must explicitly include the following sections, and feel free to add any other sections that may strengthen your report:
 - **System Design:** A detailed explanation of the specific data structures used to build your recursive parser and your State Dictionary.
 - **Testing suite:** You must also show test files demonstrating both successful compilation and your compiler's ability to gracefully catch lexical and syntax errors.

- **Limitations:** An honest discussion of observed drawbacks, failure cases, or scalability limits.
- **Collaboration:** How you collaborate effectively with your teammates.
- **Live Demonstration:** Each group will participate in a short (8-minute) live presentation and Q&A session with the teaching team. You will demonstrate your working compiler pipeline, show how it handles both valid and invalid LogicScript files, and answer technical questions regarding your engineering choices.